RESEARCH ARTICLE

A Lightweight Two-Stage Intrusion Detection System for Resource-Constrained IoT Networks Using Inter-Arrival-Time Dispersion, Mamdani Fuzzy Inference and INT8-Quantised Multiclass Attack Identification

Author One[a], Author Two[a] and Author Three[a]

[a]Department of Computer Science and Engineering, Your University, City, Country

ARTICLE HISTORY
Compiled October 27, 2026

ABSTRACT
Internet of Things (IoT) networks are exposed to a wide range of attacks, including denial-of-service (DoS), distributed DoS (DDoS), botnet floods, spoofing, reconnaissance and web-application exploits, while the devices themselves have limited memory, computation and energy budgets. Supervised multiclass intrusion detection systems (IDS) can attribute traffic to a specific attack type, but they are expensive to run continuously and tend to misclassify previously unseen (zero-day) attacks as one of the known classes. This paper proposes a lightweight two-stage IDS. Module 1 is an unsupervised timing-channel anomaly filter that analyses packet inter-arrival time (IAT) over overlapping sliding windows, computes the coefficient of variation (CV) of IAT, the mean packet rate and the relative IAT deviation from a clean normal baseline, and fuses them in a 12-rule Mamdani fuzzy inference system that outputs a continuous attack-risk score in $[0, 1]$. Only windows that Module 1 flags as suspicious or attack are forwarded to Module 2, a multiclass attack identification stage built on 19 flow features, which compares a pure-NumPy Random Forest with an LSTM network quantised to full-integer INT8 using TensorFlow Lite. Identified classes are mapped to attack-specific mitigation actions. On the CICIoT2023 dataset, with the entire Mirai botnet family held out as a zero-day proxy, the fuzzy detector achieves 82.24% accuracy, 85.69% precision, 89.60% recall and an F1-score of 87.60%, against 6.51% F1 for the adaptive Median+MAD threshold and 0% for a fixed CV threshold, and detects 75.88% of held-out Mirai windows. The Random Forest attributes known attacks with 94.25% accuracy and 91.77% macro-F1, and INT8 quantisation reduces LSTM inference latency from 51.07 ms to 0.087 ms per sequence. Because 26.77% of the windows bypass Module 2, the heavier classifier does not run on every window. The experiments also show that softmax-confidence rejection alone detects none of the unseen Mirai traffic, so the timing-based first stage is needed for zero-day coverage.

KEYWORDS
Internet of Things; intrusion detection; inter-arrival time; sliding window; coefficient of variation; fuzzy logic; zero-day attack; INT8 quantisation; CICIoT2023; edge computing

CONTACT Author One. Email: author@example.edu

1. Introduction

The Internet of Things (IoT) has become one of the fastest-growing technologies of the last decade, and industry, healthcare, transport and home automation increasingly depend on it [**?** **?**]. IoT devices usually communicate over simple, lightly protected networks, which raises security and privacy concerns [**?**]. They are also resource-constrained: limited memory, processing power and battery capacity mean they cannot host large models or heavyweight security software [**?**].

These properties make IoT networks attractive targets for a range of cyberattacks [**?**]:

- **DoS/DDoS**: a device or network resource is overwhelmed with excessive traffic [**?**].
- **Botnets (e.g. Mirai)**: a network of compromised devices is used to launch coordinated malicious activity.
- **Brute force**: repeated attempts to guess credentials and gain unauthorised access [**?**].
- **Spoofing**: an attacker impersonates a legitimate or trusted source [**?**].
- **Reconnaissance and scanning**: probing to discover open ports, services and vulnerabilities.

These attacks affect availability, authentication, network resources and communication between connected devices. IoT networks therefore need continuous monitoring and detection that can also support an appropriate mitigation response.

Intrusion Detection Systems (IDS) monitor network traffic, separate normal from malicious behaviour and raise alerts when they detect suspicious activity [**?**]. Signature- and rule-based IDS can only recognise attack patterns described in advance [**?**]. Machine learning (ML) and deep learning (DL) IDS learn patterns from traffic instead [**?**]. Binary ML-IDS label traffic only as normal or malicious and say nothing about the attack type [**?**]. Different attacks need different responses, so a multiclass IDS that names the attack category is more useful operationally. Multiclass ML/DL models are, however, computationally and memory-intensive, which makes them hard to deploy on IoT hardware [**?** **?**]. Quantisation, which lowers the numerical precision of model parameters and activations, is one of the most effective ways to cut model size and inference cost while keeping detection performance [**?** **?**].

Resource efficiency does not solve the problem of previously unseen, or zero-day, attacks [**?**]. Supervised IDS learn from known traffic, and when traffic behaves differently from anything in the training set they frequently force it into one of the known classes [**?**]. Anomaly detection, which models normal behaviour and flags departures from it, complements attack classification. Packet inter-arrival time (IAT), the interval between two consecutive packet arrivals, is a particularly useful signal for this: it is cheap to compute, contains no payload or addressing information, and changes markedly under automated attacks [**?**]. Flooding tools emit packets at a rigid clock rate, so IAT dispersion collapses ($CV \to 0$). Scanners and slow probes introduce irregular gaps, so IAT dispersion and deviation become large.

Based on these observations, this paper proposes a two-stage, resource-aware multiclass intrusion detection and mitigation pipeline for IoT networks (Figure **??**). Its contributions are:

(1) An **IAT-based anomaly detection module** that computes the CV of IAT, the mean packet rate and the relative IAT deviation over overlapping sliding

windows, and fuses them in an interpretable 12-rule Mamdani fuzzy inference system with membership functions derived from clean normal traffic. The module captures both high-dispersion (scan) and low-dispersion (flood) anomalies, which a single upper-tail CV threshold cannot.

(2) An **anti-poisoning adaptive Median+MAD threshold**, evaluated as a baseline, whose history is updated only from windows that are confidently normal.

(3) A **multiclass attack identification module** that runs only on windows flagged by Module 1. It compares a non-sequential Random Forest with an LSTM that is quantised to full-integer INT8 TensorFlow Lite for edge deployment.

(4) A **leakage-safe zero-day protocol** on CICIoT2023 in which the whole Mirai botnet family is excluded from baselines, membership functions, thresholds, scalers and classifier training.

(5) An **attack-specific mitigation mapping** and an end-to-end analysis of how much classifier computation the first-stage filter saves.

The rest of the paper is organised as follows. Section **??** reviews related work. Section **??** describes the proposed methodology. Section **??** gives the dataset and experimental setup. Section **??** presents and discusses the results. Section **??** covers limitations, and Section **??** concludes.

2. Literature Survey

Temporal and privacy-preserving features. Many IDS rely on packet attributes such as MAC addresses, packet lengths and source/destination IP addresses, which carry sensitive information and raise privacy concerns. Ref. [**?**] showed that IAT-based temporal features can detect stealthy slow DoS and spoofing attacks without inspecting such identifiers. Later work has combined IAT with deep and reinforcement learning, but evaluations are often limited to simulated laboratory settings, and most studies stop at detection with little attention to mitigation. Temporal NetFlow features were also shown to improve ML-based NIDS in [**?**].

Sliding-window multiclass detection. Ref. [**?**] segments traffic into temporal sliding windows and compares deep convolutional feature extractors with handcrafted summary statistics. It finds that simple statistical descriptors with tree ensembles such as XGBoost can outperform deep models. Computing more than a hundred statistics per window, however, adds latency that is impractical on constrained devices, and the supervised models remain vulnerable to zero-day attacks and include no mitigation.

Anomaly analysis with heavy pipelines. Ref. [**?**] identifies compromised nodes by combining statistical outlier rejection, time-frequency transforms, deep autoencoders and swarm-optimised classifiers over sliding-window statistics. Detection accuracy is reasonable, but the multi-stage pipeline has large computational and memory costs, degrades at high attack intensities, and only flags infected nodes without countermeasures.

Edge-friendly models. Boosted Convolutional Trees (BCT) [**?**] combine 1D-convolutional feature extraction, soft decision trees and boosting. Evaluated on CICIoT2023, X-IIoTID and Edge-IIoTset, BCT reaches competitive accuracy with a smaller model and faster inference than CNN/LSTM baselines, but it is sensitive to class imbalance and was not validated on physical devices.

Table 1. Comparison of representative related work with the proposed system.

Work	Multiclass	Lightweight / quantised	Zero-day	Interpretable	Mitigation
Temporal IAT [**?**]	–	✓	partial	–	–
Hybrid fusion [**?**]	✓	–	–	partial	–
Anomaly analysis [**?**]	–	–	partial	–	–
BCT [**?**]	✓	✓	–	–	–
DNN-BiLSTM+QAT [**?**]	✓	✓	–	–	–
Dynamic 3σ [**?**]	–	✓	✓	✓	–
Energy-aware DL [**?**]	✓	✓	–	–	–
Proposed	✓	✓	✓	✓	✓

Quantised lightweight IDS. Ref. [**?**] combines RAL-MIFS feature selection, two-stage IPCA, a DNN-BiLSTM classifier, quantisation-aware training (QAT), post-training dynamic quantisation (PTDQ) and Optuna tuning on CIC-IDS2017 and CICIoT2023. It reports high accuracy with a large reduction in model size, but minority classes remain difficult and the system was not evaluated on real constrained hardware. A quantised autonomous edge IDS for the Internet of Medical Things is presented in [**?**].

Dynamic statistical thresholds. Ref. [**?**] proposes a lightweight dynamic-threshold detector for edge devices that uses a 30-second window, the 3σ rule, a Kolmogorov–Smirnov normality check and protocol-aware filtering on just two features (packets/s and bytes/s). It reaches 94.7–96.2% accuracy at 5.1% CPU usage on CICIDS2017, NSL-KDD and simulated industrial traffic, but it assumes approximately normal traffic and struggles with non-stationary patterns.

Energy-aware DL. Ref. [**?**] combines SeparableConv1D layers, SMOTE, structured pruning and QAT. Deployed on a Raspberry Pi 4, it reaches 86.2% accuracy on NSL-KDD with 94,408 parameters, 3.6× faster inference and 60% lower power than LSTM models. It still misses many attacks, and its generalisation to mixed IoT traffic is unclear.

Open-set recognition. Open-set IDS [**?**] detect unknown classes through distance- or confidence-based rejection. Our results (Section **??**) show that confidence rejection on top of a closed-set classifier can be unreliable for floods that resemble known classes.

Research gap. Existing work tends to optimise one of three goals: detection accuracy, resource efficiency or zero-day robustness. Few systems combine an interpretable, label-free timing filter with a quantised multiclass classifier that runs only when needed, and fewer still connect the identified class to a specific mitigation action. Table **??** summarises this comparison.

3. Proposed Methodology

The proposed system has two modules connected by a gating decision (Figure **??**). Module 1 decides whether a traffic window is anomalous. Module 2 runs only for anomalous windows and determines the attack type, which then selects a mitigation action. After mitigation, Module 1 keeps monitoring the same timing statistics, so operators can check that the IAT dispersion returns to its normal range.

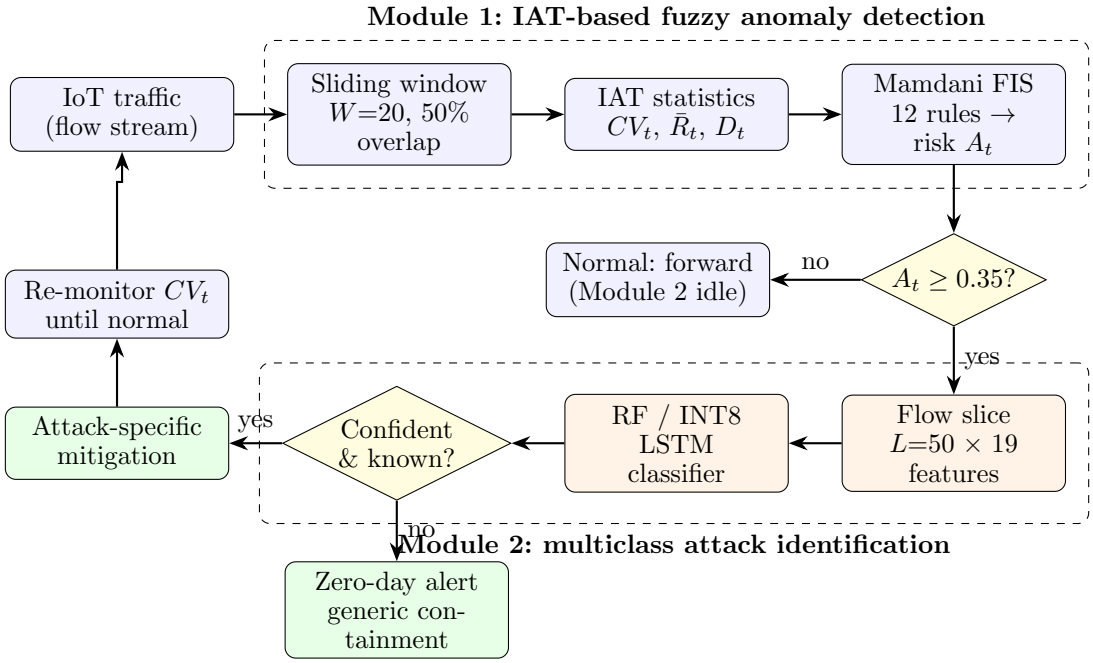


Figure 1. Architecture of the proposed two-stage IDS. Module 1 screens every window cheaply; Module 2 is only invoked for suspicious or attack windows, and its output drives the attack-specific mitigation and re-monitoring loop.

3.1. *Module 1: IAT-Based Anomaly Detection*

3.1.1. Packet inter-arrival time

The IAT of the i-th packet is the time between its arrival and the arrival of the previous packet:

$$\mathrm{IAT}_i = t_i - t_{i-1}, \tag{1}$$

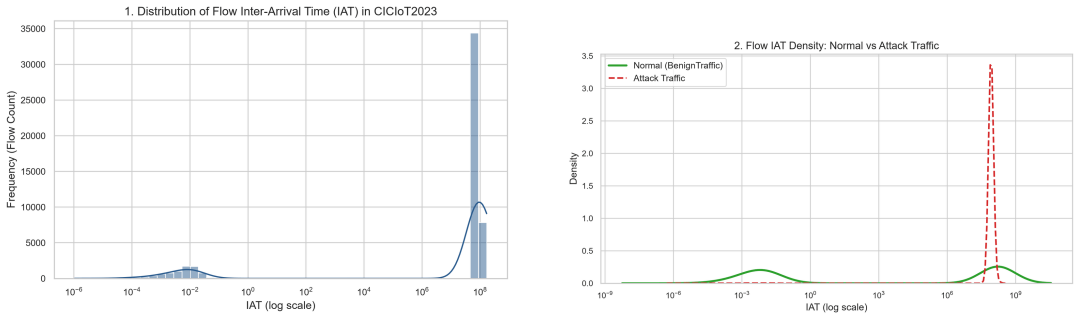
where t_i is the arrival timestamp of packet i. In a live deployment, Eq. (**??**) is computed directly at the gateway. The public CSV release of CICIoT2023 does not provide packet timestamps but does provide an `IAT` feature extracted by the capture tool. We use this feature directly and do not synthesise or back-calculate timestamps, so the evaluation does not depend on fabricated timing.

3.1.2. Overlapping sliding windows

The observation stream is divided into windows of W consecutive records with step $S = W(1-\rho)$, where ρ is the overlap. We use $W = 20$ and $\rho = 0.5$ ($S = 10$), so window t covers records $[tS,\, tS+W-1]$. Each window's ground truth is derived from its attack fraction,

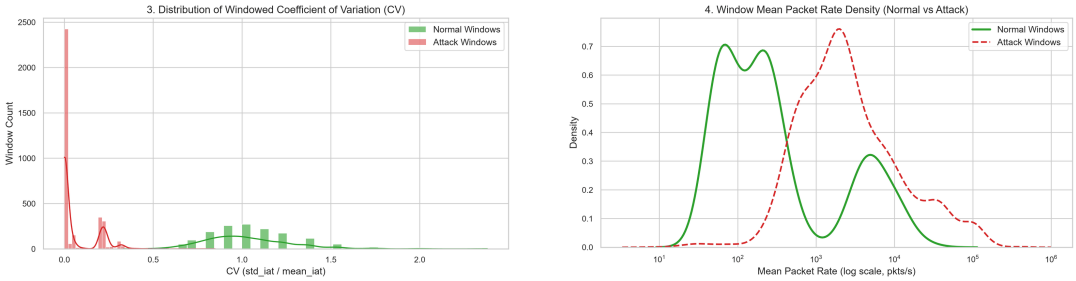
$$\phi_t = \frac{1}{W}\sum_{i\in t} \mathbf{1}(y_i \neq \texttt{BenignTraffic})\,, \qquad Y_t = \mathbf{1}(\phi_t \geq 0.5), \tag{2}$$

and the dominant class label is kept for cohort-level analysis.



(a) IAT distribution (b) IAT: normal vs. attack

Figure 2. Distribution of the packet inter-arrival time feature in CICIoT2023 and its separation between benign and attack traffic.



(a) Window CV of IAT (b) Window mean packet rate

Figure 3. Window-level CV and packet-rate distributions. Many attack windows have CV values below the benign mode. Upper-tail CV tests cannot detect these windows.

3.1.3. Window-level statistical features

For each window we compute the mean and standard deviation of IAT,

$$\mu_t = \frac{1}{W}\sum_{i=1}^{W} \mathrm{IAT}_i, \qquad \sigma_t = \sqrt{\frac{1}{W}\sum_{i=1}^{W} (\mathrm{IAT}_i - \mu_t)^2}, \tag{3}$$

and the dimensionless coefficient of variation

$$CV_t = \frac{\sigma_t}{\mu_t + \epsilon}, \qquad \epsilon = 10^{-12}. \tag{4}$$

Two further features add volumetric and baseline-relative information: the mean packet rate $\bar{R}_t = \frac{1}{W}\sum_i \mathrm{Rate}_i$, and the relative IAT deviation from the clean normal baseline,

$$D_t = \frac{|\mu_t - \mu_{\mathrm{N}}|}{\mu_{\mathrm{N}}}, \tag{5}$$

where μ_{N} is the mean IAT of benign training traffic only. Benign IoT traffic shows Poisson-like jitter with $CV \approx 1$. Automated floods emit packets at a near-constant clock, so $CV \to 0$. Scans and probes produce irregular sweeps, so CV and D are large. Figures **??**–**??** show these distributions in the data.

Table 2. Data-driven membership functions of the Mamdani FIS.

Variable	LOW	MEDIUM	HIGH
CV_t (0–6)	trimf(0, 0, 0.65)	trimf(0.50, 1.02, 1.55)	trapmf(1.40, 1.80, 6, 6)
$\bar{R}_t$ (pkt/s)	trimf(0, 0, 350)	trimf(200, 600, 1500)	trapmf(1000, 2500, $5{\times}10^4$, $5{\times}10^4$)
D_t (0–5)	trimf(0, 0, 0.18)	trimf(0.12, 0.35, 0.65)	trapmf(0.50, 0.90, 5, 5)
Risk A_t (0–1)	trimf(0, 0, 0.35)	trimf(0.25, 0.50, 0.75)	trapmf(0.60, 0.85, 1, 1)

3.1.4. Adaptive Median+MAD threshold with anti-poisoning

An adaptive threshold is computed from a history H_t of CV values taken from previously accepted normal windows:

$$M_t = \text{Median}(H_t), \tag{6}$$

$$\text{MAD}_t = \text{Median}\left(|CV_j - M_t| : CV_j \in H_t\right), \tag{7}$$

$$T_t = M_t + k \cdot \text{MAD}_t, \tag{8}$$

with sensitivity $k = 3$ and a bounded history buffer of 500 windows. If the history is updated with every window, as in a naive adaptive threshold, attack windows are absorbed into H_t, the threshold widens and the detector gradually becomes blind (baseline poisoning). We therefore update the history only when the window is confidently normal:

$$H_{t+1} = \begin{cases} H_t \cup \{CV_t\}, & CV_t \le T_t \ \wedge \ A_t < 0.35, \\ H_t, & \text{otherwise}, \end{cases} \tag{9}$$

where A_t is the fuzzy attack risk defined below. Figure **??** shows the CV timeline and the adaptive threshold.

3.1.5. Mamdani fuzzy inference

Thresholding a single statistic cannot capture anomalies in both directions. We therefore fuse three inputs, CV_t, $\bar{R}_t$ and D_t, in a Mamdani fuzzy inference system (FIS):

$$A_t = f_{\text{FIS}}\left(CV_t,\ \bar{R}_t,\ D_t\right) \in [0, 1]. \tag{10}$$

Each input has three linguistic terms (LOW, MEDIUM, HIGH) with triangular or trapezoidal membership functions. Their boundaries are set from percentiles of the clean benign baseline, not chosen by hand (Table **??**, Figure **??**). For example, the benign CV median is 1.00 with MAD 0.106, its 25th–75th percentile range is 0.82–1.11, and the benign median packet rate is 202 pkt/s.

The rule base (Table **??**) encodes domain knowledge. Rule R11 (LOW CV $\wedge$ HIGH rate $\Rightarrow$ HIGH risk) captures constant-rate floods such as DoS and Mirai, and rule R12 captures stealthy, low-rate scans. Rules are combined with min (AND) and max (aggregation) operators.

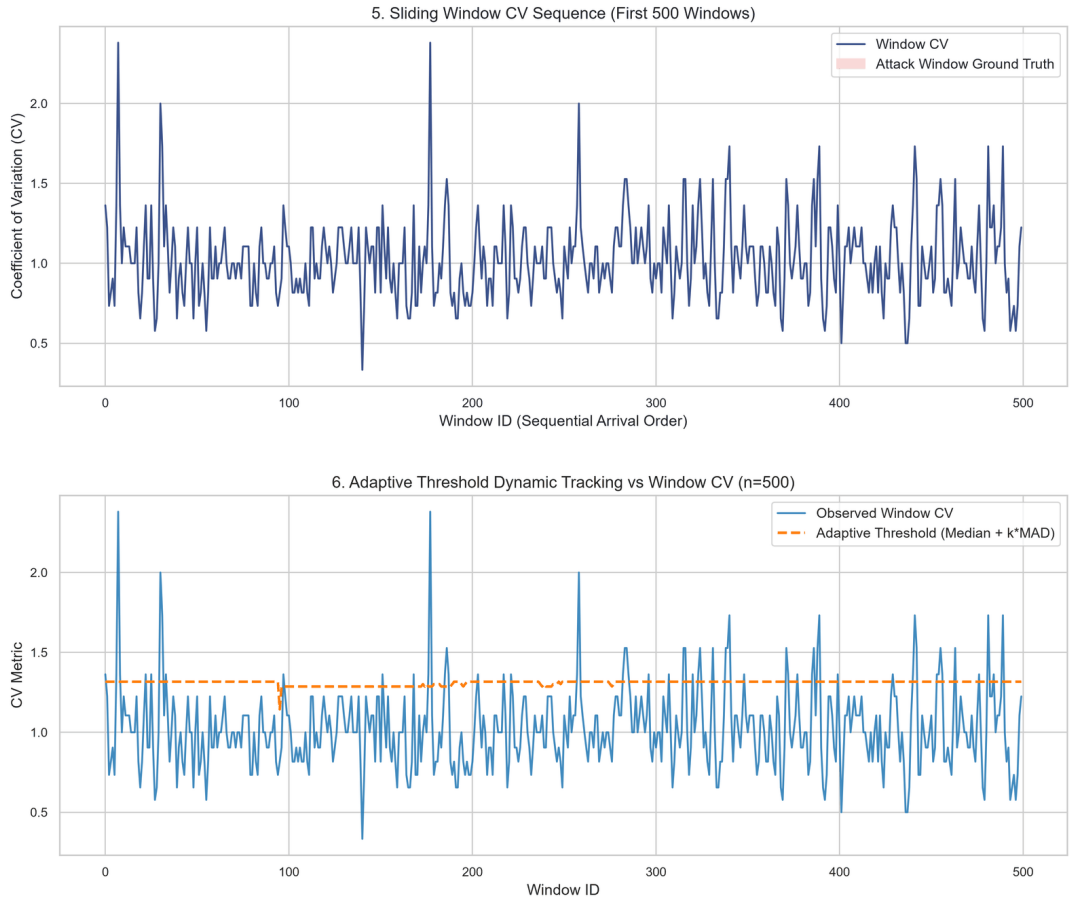


Figure 4. Top: sliding-window CV_t over the evaluation stream. Bottom: the adaptive Median+MAD threshold T_t against CV_t. Flood phases push CV_t well below the threshold, so a CV-only upper-tail test misses them.

3.1.6. Defuzzification and decision

The aggregated output fuzzy set is defuzzified with the centroid method,

$$A_t = \frac{\int z\, \mu_{\text{agg}}(z)\, dz}{\int \mu_{\text{agg}}(z)\, dz}, \tag{11}$$

and mapped to three decision zones:

$$\text{Verdict}_t = \begin{cases} \textsc{Normal}, & A_t < 0.35, \\ \textsc{Suspicious}, & 0.35 \le A_t < 0.65, \\ \textsc{Attack}, & A_t \ge 0.65. \end{cases} \tag{12}$$

Windows labelled Suspicious or Attack are forwarded to Module 2. Normal windows are released without further processing.

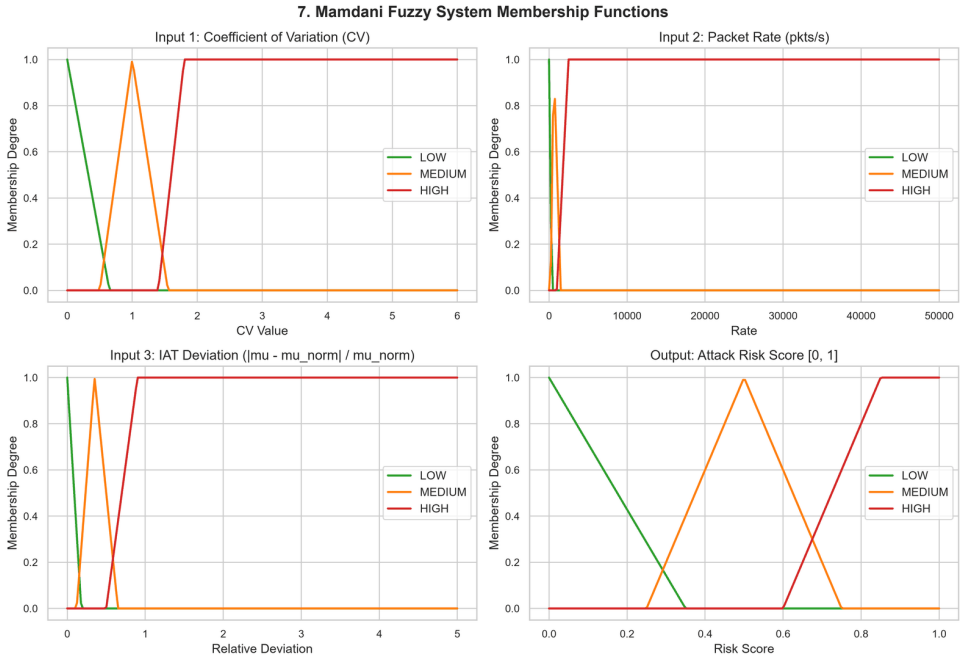


Figure 5. Membership functions of the three antecedents (CV, packet rate, IAT deviation) and the attack-risk consequent.

Table 3. Interpretable fuzzy rule base.

Rule	Antecedent	Risk	Rationale
R1	CV MED ∧ Rate LOW ∧ Dev LOW	LOW	Steady-state IoT behaviour
R2	CV LOW ∧ Rate LOW ∧ Dev LOW	LOW	Periodic low-rate beaconing
R3	CV HIGH ∧ Rate HIGH	HIGH	Volumetric burst DDoS
R4	CV HIGH ∧ Dev HIGH	HIGH	Pulse attack / recon sweep
R5	Rate HIGH ∧ Dev HIGH	HIGH	Volumetric attack with timing shift
R6	CV MED ∧ Rate HIGH	HIGH	High-rate Poisson-like flood
R7	CV HIGH ∧ Rate MED	HIGH	Irregular burst attack
R8	CV MED ∧ Rate MED ∧ Dev MED	MED	Borderline activity
R9	Rate HIGH	HIGH	Volumetric override
R10	Dev HIGH	HIGH	Timing-channel collapse
R11	CV LOW ∧ Rate HIGH	HIGH	Constant-rate flood (DoS/Mirai)
R12	CV HIGH ∧ Rate LOW	MED	Stealthy scan / slow probe

3.2. *Module 2: Multiclass Attack Identification*

3.2.1. Traffic buffering and feature selection

Module 1 uses only timing and rate statistics, so extracting the full feature set for every window would waste resources. A ring buffer holds the most recent flow records. When Module 1 flags window t, the slice of $L = 50$ consecutive flows ending at that window is retrieved. Each flow is described by 19 features chosen from the 46 available in CICIoT2023 (Table **??**). Labels, scenario identifiers and IP addresses are excluded to prevent leakage and protect privacy.

3.2.2. Leakage-safe preprocessing and sequence construction

Missing values are imputed with training-set medians, infinite values are clipped to the finite training maximum, and a `StandardScaler` fitted on training flows only is applied

Table 4. Features used by Module 2.

Group	Features
Timing & rate	IAT, Rate, Srate, Drate, flow_duration, Duration
Volume & size	Header_Length, Tot size, Tot sum, Number, AVG, Std, Min, Max
Protocol & state	Protocol Type, TCP, UDP, syn_count, ack_count

to every split. A provenance audit showed that the Kaggle CSVs are row-shuffled, so arbitrary consecutive rows mix unrelated classes. Sequences are therefore built only inside homogeneous class blocks, and the train/validation/test split (70/15/15) is made on the blocks *before* sequences are formed. This ensures that overlapping sequences never cross the split boundary. Sequences have shape $(L, F) = (50, 19)$, with step 25 for training and 15 for evaluation.

3.2.3. Classifiers

LSTM. An `LSTM(64)` layer is followed by `Dropout(0.2)`, `Dense(32, ReLU)` and a 31-way softmax. It is trained with Adam (learning rate 10^{-3}, batch 64) and sparse categorical cross-entropy, with early stopping (patience 5). The recurrence is statically unrolled over the 50 time steps to remove dynamic `TensorList` operations, which allows native full-integer TFLite conversion.

Random Forest baseline. A pure-NumPy Random Forest (30 trees, maximum depth 12, percentile split candidates) uses non-sequential inputs: the last observation of the sequence plus the per-feature mean and standard deviation over the sequence. It shows whether recurrent modelling adds value over simple aggregates.

3.2.4. INT8 post-training quantisation

The FP32 LSTM is converted to a full-integer INT8 TFLite model with a representative calibration set drawn from the training sequences. For a tensor with observed range $[r_{\min}, r_{\max}]$, affine quantisation uses

$$s = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}, \qquad z = \operatorname{round}\Big(q_{\min} - \frac{r_{\min}}{s}\Big), \tag{13}$$

$$q = \operatorname{clip}\Big(\operatorname{round}\Big(\frac{r}{s}\Big) + z,\ q_{\min},\ q_{\max}\Big), \tag{14}$$

with $q_{\min} = -128$ and $q_{\max} = 127$, and dequantisation $\hat{r} = s(q - z)$. Inference runs on the XNNPACK CPU delegate.

3.2.5. Zero-day rejection

A closed-set classifier always outputs one of its known classes. To flag unknown traffic, we apply maximum-softmax-probability (MSP) rejection,

$$\hat{Y}_t = \begin{cases} \arg\max_k p_k, & \max_k p_k \geq \tau, \\ \textsc{Unknown (zero-day)}, & \text{otherwise}, \end{cases} \tag{15}$$

with $\tau = 0.0643$ set to the 10th percentile of the confidence of correct validation predictions.

Table 5. Attack-family to mitigation mapping.

Identified family	Mitigation action
DDoS / DoS floods	Per-source rate limiting, SYN cookies, upstream traffic filtering / blackholing
Botnet (Mirai-like)	Quarantine infected device, block C&C endpoints, enforce credential reset
Reconnaissance / scans	Temporary source block, port-scan throttling, honeypot redirection
Spoofing / MITM	Dynamic ARP inspection, static ARP bindings, DNSSEC validation
Web / brute force	Account lockout, CAPTCHA/back-off, WAF signature update
Unknown (zero-day)	Rate-limit and isolate segment, capture traffic, raise operator alert

3.3. *Attack-Specific Mitigation and Re-Monitoring*

The identified class $\hat{Y}_t$ selects a mitigation action according to the attack family (Table **??**). Unknown traffic receives a conservative containment action and an operator alert. After an action is applied, Module 1 keeps computing CV_t, $\bar{R}_t$ and D_t. Mitigation is considered successful when the fuzzy risk falls back below 0.35 for a run of consecutive windows. Otherwise the response is escalated, for example from rate limiting to blocking. In the current prototype, this mapping is implemented as a decision policy in the dashboard; enforcing actions in the network data plane is left for future work.

4. Dataset and Experimental Setup

4.1. *CICIoT2023 dataset*

CICIoT2023 is a large-scale IoT benchmark captured from a topology of 105 IoT devices. We use its public CSV release, which contains 6,668,822 flows across 34 classes: `BenignTraffic` plus DDoS and DoS floods, the Mirai botnet family, reconnaissance and vulnerability scans, spoofing/MITM, and web/application attacks (SQL injection, command injection, XSS, browser hijacking, dictionary brute force, back-door malware and uploading attacks).

4.2. *Zero-day protocol*

All three Mirai classes (`Mirai-greeth_flood`, `Mirai-greip_flood` and `Mirai-udpplain`) are held out as a zero-day proxy. They are excluded from the benign baseline, the membership-function boundaries, the threshold calibration, the scaler and label encoder, and classifier training, and are introduced only in the final test stream. Module 2 therefore covers 31 known classes. Holding out an entire family is an empirical proxy for novel attacks, not a guarantee against all future exploits.

4.3. *Evaluation stream and metrics*

Module 1 is evaluated on a phased stream of 50,000 flows, which gives 4,999 windows: 1,499 normal, 2,499 known-attack and 999 held-out Mirai windows. Three detectors are compared on the same stream: (i) a *simple CV threshold* (flag if CV_t exceeds the 95th percentile of benign CV); (ii) the *adaptive Median+MAD* threshold with anti-

Table 6. Anomaly detection performance on 4,999 windows.

Method	Accuracy	Precision	Recall	F1-score	FPR	Mirai det.
Simple CV threshold	28.49%	0.00%	0.00%	0.00%	**4.94%**	0.00%
Adaptive Median+MAD	29.35%	44.40%	3.51%	6.51%	10.28%	11.61%
Fuzzy (proposed)	**82.24%**	**85.69%**	**89.60%**	**87.60%**	34.98%	**75.88%**

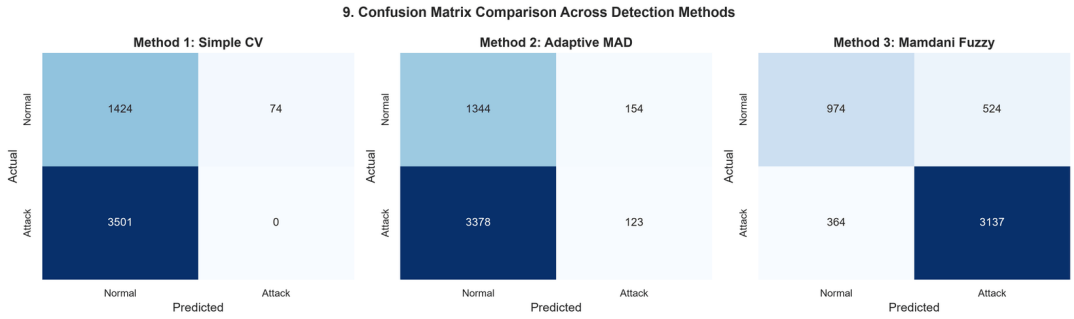


Figure 6. Window-level confusion matrices of the simple CV, adaptive MAD and fuzzy detectors (TN/FP/FN/TP of 1424/74/3501/0, 1344/154/3378/123 and 974/524/364/3137 respectively).

poisoning; and (iii) the proposed *fuzzy* system, where a window counts as detected when $A_t \geq 0.35$. We report accuracy, precision, recall (TPR), F1-score and false-positive rate (FPR). Module 2 is evaluated on held-out test sequences with accuracy, macro-F1, weighted-F1, mean and P95 CPU latency per sequence, and model size. All experiments use random seed 42 and run on a desktop Intel Core i7 CPU with Python, scikit-fuzzy and TensorFlow/TFLite.

5. Results and Discussion

5.1. *Anomaly detection performance (Module 1)*

Table **??** and Figure **??** compare the three detectors. The simple CV threshold detects no attack windows (recall 0%), and the adaptive MAD threshold reaches only 3.51% recall. Both methods test the upper tail ($CV_t > T$), but the dominant flood attacks in CICIoT2023 *lower* IAT dispersion (often $CV \approx 0.002$), so these windows fall below any upper threshold. The fuzzy system fuses dispersion, rate and baseline deviation and reaches 82.24% accuracy, 85.69% precision, 89.60% recall and 87.60% F1. Its main cost is a 34.98% false-positive rate, mostly from bursty benign windows assigned to the Suspicious band. These windows are not dropped: they are passed to Module 2 for verification.

Table **??** breaks the alarm rate down by cohort. The fuzzy detector flags 95.08% of known-attack windows and 75.88% of held-out Mirai windows, against 0% and 11.61% for the best threshold baseline. The mean fuzzy risk is 0.37 for normal windows, 0.79 for known attacks and 0.59 for unseen Mirai traffic. The risk timeline in Figure **??** shows that the score rises clearly when attack phases begin.

5.1.0.1. Sensitivity analysis.. Figure **??** varies the window size and the MAD multiplier. Accuracy peaks at $W = 20$ (0.823), compared with 0.727 at $W = 10$, 0.807 at $W = 50$ and 0.750 at $W = 100$. Small windows are noisy, and large windows blur the boundaries between attack phases and delay alerts. For the adaptive MAD

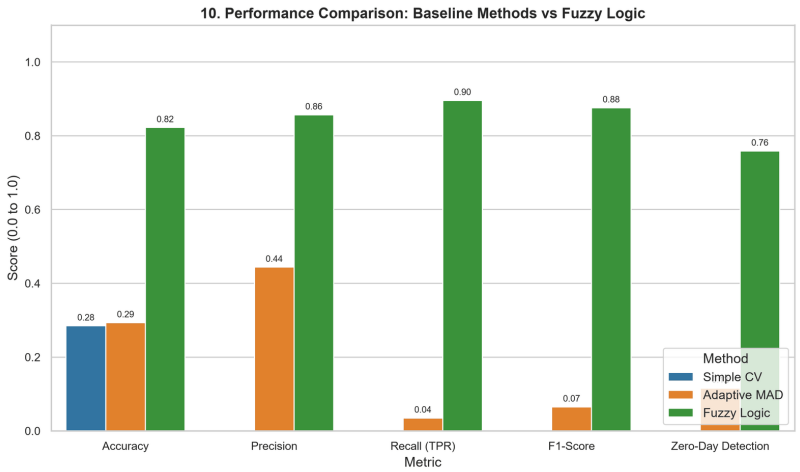


Figure 7. Accuracy, precision, recall, F1 and FPR of the three anomaly detectors.

Table 7. Alarm rate per traffic cohort.

Cohort (windows)	Simple CV	Adaptive MAD	Fuzzy	Mean risk A_t
Normal (1,499)	4.94%	13.54%	34.96%	0.374 ± 0.290
Known attacks (2,499)	0.00%	15.73%	**95.08%**	0.792 ± 0.162
Held-out Mirai (999)	0.00%	11.61%	**75.88%**	0.595 ± 0.278

baseline, a larger k lowers accuracy steadily (0.280 at $k = 2$ to 0.177 at $k = 4$), which confirms that tuning k cannot fix its one-sided test.

5.2. *Multiclass classification performance (Module 2)*

Table **??** and Figure **??** compare the classifiers on 87 held-out test sequences covering the 31 known classes. The Random Forest performs best, with 94.25% accuracy, 91.77% macro-F1 and 93.02% weighted-F1, and perfect per-class F1 on most flood, spoofing and web classes. Its weakest classes are `Recon-OSScan` (F1 0.50), `Recon-PortScan` (0.57) and `DDoS-UDP_Flood` (0.57), along with the very rare `Recon-PingSweep` and `Backdoor_Malware`, which have only one test sequence each. The compact LSTM reaches only 34.48% accuracy under the small training budget used here (429 training sequences, a few epochs; Figure **??**). Flow-level volumetric aggregates are thus already highly discriminative on CICIoT2023, and the recurrent model needs considerably more data and training to learn temporal transitions across 31 classes.

5.3. *Quantisation and computational efficiency*

Full-integer INT8 quantisation keeps the LSTM's accuracy unchanged (34.48%), slightly raises macro-F1 (20.47% to 21.73%) and gives 85.06% prediction agreement with the FP32 model (Figure **??**). Mean latency falls from 51.07 ms to 0.087 ms per sequence, a speed-up of about 587×, and P95 latency falls from 55.10 ms to 0.099 ms. Most of this gain comes from replacing the Keras eager runtime with the XNNPACK-accelerated integer TFLite interpreter, not from arithmetic precision alone. The model file size is the same in both formats (0.31 MB) because the network has few param-

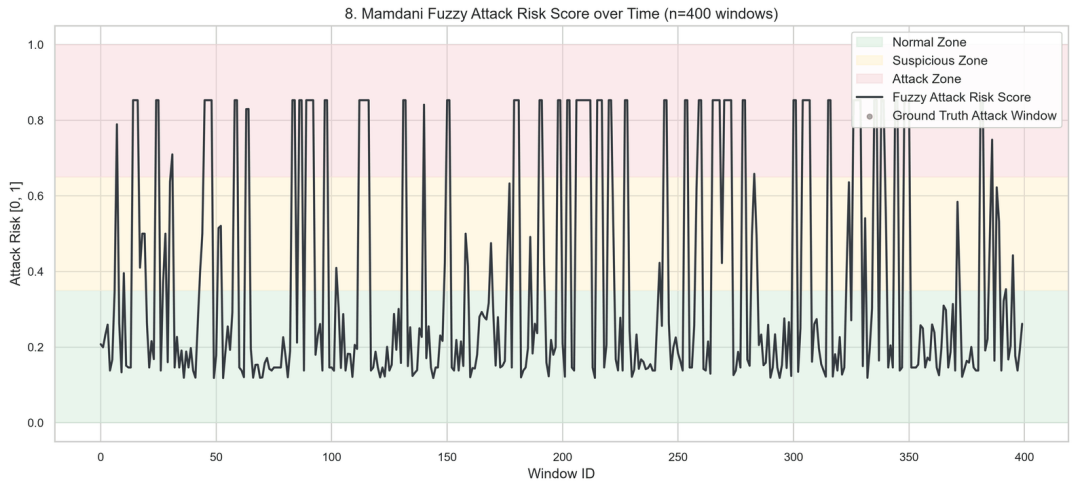
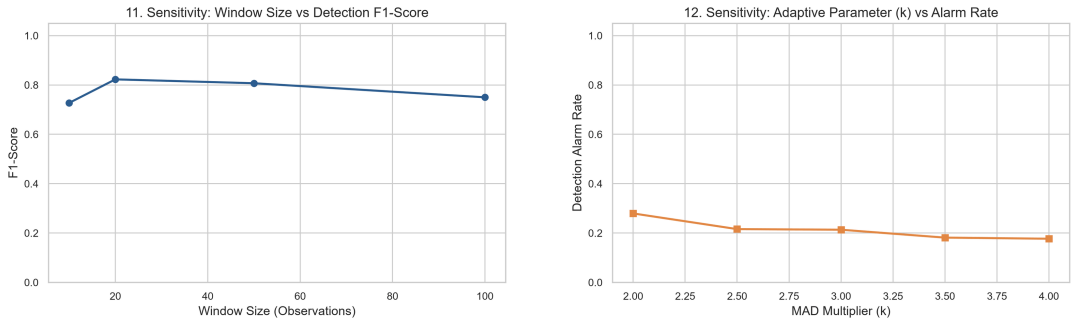


Figure 8. Continuous fuzzy attack-risk score over the evaluation stream with the 0.35 and 0.65 decision boundaries.



(a) Window size W

(b) MAD multiplier k

Figure 9. Sensitivity of detection performance to the sliding-window size and to the adaptive-threshold multiplier.

eters and the file size is dominated by graph structure and metadata. Larger models would show the expected weight-memory reduction of about 75%.

The two-stage design also saves computation at the system level (Table **??**). Of the 4,999 windows, 26.77% were judged NORMAL by Module 1 and never reached Module 2, so the classifier ran on only 73.23% of the windows. The evaluation stream is deliberately attack-heavy. In operational IoT networks, where benign traffic usually dominates, the share of windows that bypass Module 2 would be much larger.

5.4. *Held-out (zero-day) attack analysis*

The two modules behave very differently on the 111 held-out Mirai sequences. MSP rejection in Module 2 detects none of them (0% zero-day detection). The unseen Mirai floods receive a *higher* mean confidence (0.202) than known traffic (0.119), and all accepted samples are assigned to similar known classes: 73 to `DDoS-UDP_Fragmentation` and 38 to `DoS-UDP_Flood`. At the chosen threshold, MSP also wrongly rejects 35.63% of known traffic. Module 1 does not depend on any attack signature and flags 75.88% of the Mirai windows because of their abnormal timing and rate. Supervised confi-

Table 8. Multiclass attack identification on the known-class test set.

Model	Acc.	Macro-F1	Wtd-F1	Lat. mean (ms)	Lat. P95 (ms)	Size (MB)
Random Forest	**94.25%**	**91.77%**	**93.02%**	0.196	0.230	2.68
FP32 LSTM (Keras)	34.48%	20.47%	21.18%	51.070	55.096	0.31
INT8 LSTM (TFLite)	34.48%	21.73%	22.48%	**0.087**	**0.099**	0.31

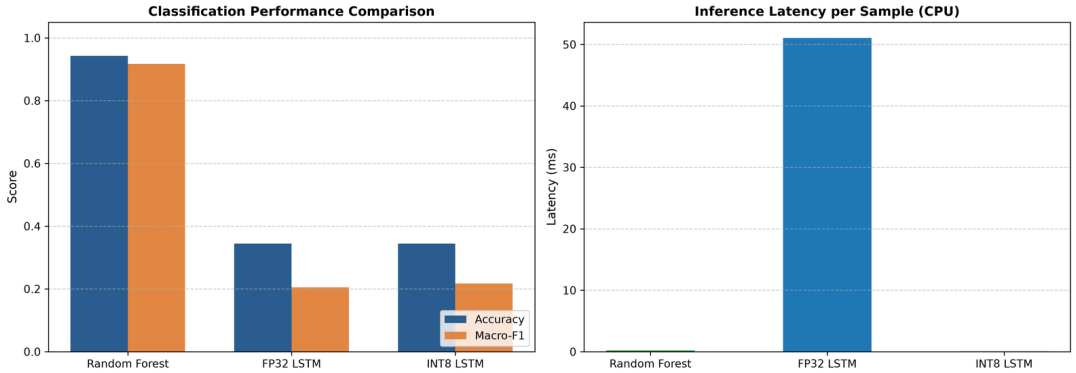


Figure 10. Accuracy/F1, latency and model size of the Random Forest, FP32 LSTM and INT8 LSTM.

dence alone is therefore not a reliable zero-day detector, and the label-free timing filter provides most of the system's zero-day coverage. Combining the two, for example by alerting on windows that Module 1 rates as high-risk but Module 2 assigns with low class-consistency, is a natural extension.

5.5. *Interactive prototype*

The pipeline is also available as an interactive Streamlit dashboard (Figure **??**). It lets an analyst enter window statistics or choose attack presets, view the activated fuzzy rules and membership degrees, and see the resulting verdict and recommended mitigation.

6. Limitations

(1) **No packet timestamps.** The CSV release of CICIoT2023 omits raw timestamps, so the timing analysis uses the flow-level `IAT` feature and assumes the record order reflects arrival order. Validation on raw PCAP captures is required.
(2) **False positives.** The 34.98% FPR comes mainly from legitimate high-throughput bursts such as firmware updates or streaming. Module 2 verifies these windows, but adding application-layer entropy features or a persistence term to the FIS would reduce the alert load.
(3) **LSTM training budget.** The LSTM was trained on a small sample budget (600 flows per class), which explains its low accuracy. Its results should be read as a quantisation and latency study rather than as an upper bound on recurrent accuracy.
(4) **Slow-rate attacks.** Very slow attacks, such as Slowloris with minute-scale gaps, need much longer windows ($W \geq 200$) to accumulate enough statistical deviation.

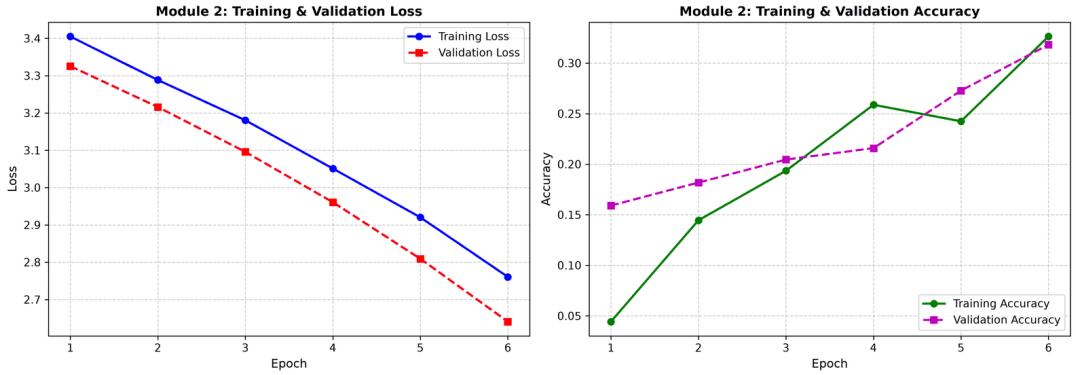
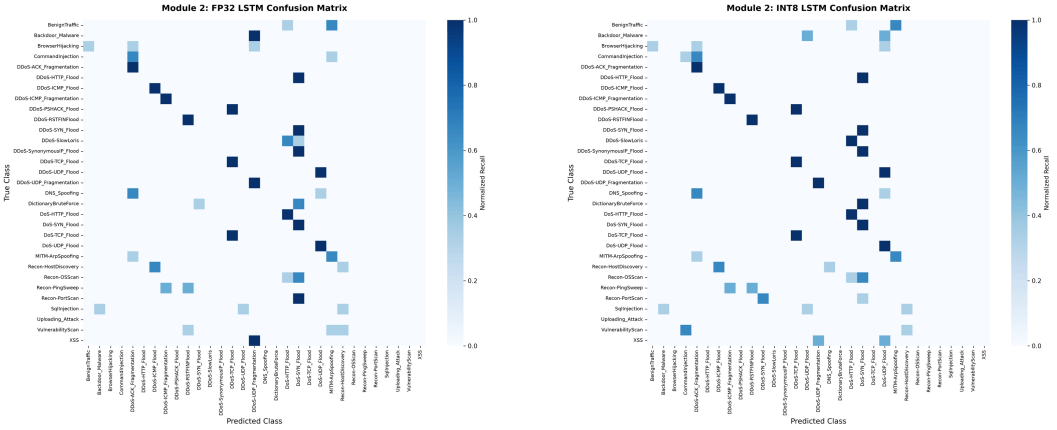


Figure 11. Training and validation loss and accuracy of the LSTM.



(a) FP32 LSTM

(b) INT8 LSTM

Figure 12. Normalised confusion matrices of the LSTM before and after INT8 quantisation.

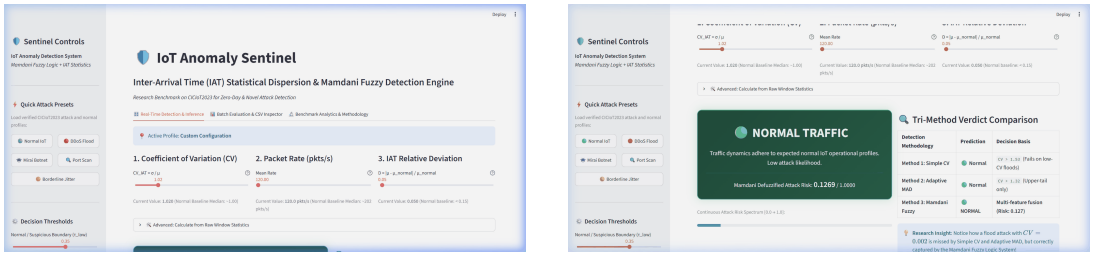
(5) **Hardware and mitigation.** Latency was measured on a desktop CPU. ARM Cortex-A72 (Raspberry Pi 4) latency is expected to be about 1–3 ms per sequence but has not yet been measured. The mitigation mapping has not yet been enforced or evaluated in a live testbed.

7. Conclusion and Future Work

This paper presented a lightweight two-stage intrusion detection system for resource-constrained IoT networks. The first stage turns packet inter-arrival time into three interpretable window statistics, the coefficient of variation, the packet rate and the deviation from a clean baseline, and fuses them with a 12-rule Mamdani fuzzy system. On CICIoT2023, the fuzzy detector improves recall from 3.51% (adaptive MAD) to 89.60% and F1 from 6.51% to 87.60%, mainly because it detects low-dispersion floods that upper-tail CV thresholds cannot, and it flags 75.88% of a fully held-out Mirai botnet family. The second stage runs only on flagged windows. It identifies 31 known attack classes with 94.25% accuracy using a compact Random Forest, and INT8 quantisation reduces LSTM inference latency to under 0.1 ms. The study also shows that softmax-confidence rejection fails completely on unseen Mirai floods, which supports

Table 9. End-to-end gating of Module 2 by Module 1.

Module 1 verdict	Windows	Share	Module 2 invoked
Normal	1,338	26.77%	No
Suspicious	489	9.78%	Yes
Attack	3,172	63.45%	Yes
Total	4,999	100%	73.23%



(a) Dashboard view

(b) Verdict and rule explanation

Figure 13. Streamlit prototype of the proposed IDS.

combining a label-free timing filter with supervised attribution.

Future work will (i) replay raw PCAP traces to compute IAT from true packet timestamps; (ii) add a persistence input and adaptive membership functions to reduce false positives; (iii) train the LSTM, or a temporal convolutional network, on the full dataset and use distance-based open-set rejection; (iv) deploy the INT8 pipeline on Raspberry Pi and ESP32-class gateways to measure latency and energy; and (v) enforce the attack-specific mitigation actions through SDN or firewall rules and measure how quickly IAT dispersion returns to normal after each action.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Data availability statement

The CICIoT2023 dataset is publicly available from the Canadian Institute for Cybersecurity. Code, trained models and result files are available from the authors on reasonable request.

References

[1] Jing, Q.; Vasilakos, A.V.; Wan, J.; Lu, J.; Qiu, D. Security of the Internet of Things: perspectives and challenges, *Wireless Networks* **2014**, *20*(8), 2481–2501.

[2] Benkhelifa, E.; Welsh, T.; Hamouda, W. A critical review of practices and challenges in intrusion detection systems for IoT: Toward universal and resilient systems, *IEEE Communications Surveys & Tutorials* **2018**, *20*(4), 3496–3509.

[3] Luay, M.; Layeghy, S.; Noorbin, N.; Sarhan, M.; Kulatilleke, G.; Moustafa, N.; Portmann, M. Time matters: Temporal NetFlow features for ML-based network intrusion detection, *IEEE Access* **2026**.

[4] Labonne, M. Anomaly-based network intrusion detection using machine learning. Ph.D. Thesis, Institut Polytechnique de Paris, 2020.
[5] Nguyen, X.H.; Nguyen, X.D.; Huynh, H.H.; Le, K.H. Realguard: A lightweight network intrusion detection system for IoT gateways, *Sensors* **2022**, *22*(2), 432.
[6] Rekha, S.; Thirupathi, L.; Renikunta, S.; Gangula, R. Study of security issues and solutions in Internet of Things (IoT), *Materials Today: Proceedings* **2023**, *80*, 3554–3559.
[7] Yang, Y.; Wu, L.; Yin, G.; Li, L.; Zhao, H. A survey on security and privacy issues in Internet-of-Things, *IEEE Internet of Things Journal* **2017**, *4*(5), 1250–1258.
[8] Hamad, S.A.; Sheng, Q.Z.; Zhang, W.E.; Nepal, S. Realizing an internet of secure things: A survey on issues and enabling technologies, *IEEE Communications Surveys & Tutorials* **2020**, *22*(2), 1372–1391.
[9] Pakmehr, A.; Aßmuth, A.; Taheri, N.; Ghaffari, A. DDoS attack detection techniques in IoT networks: a survey, *Cluster Computing* **2024**, *27*(10), 14637–14668.
[10] Volety, T.; Saini, S.; McGhin, T.; Liu, C.Z.; Choo, K.K.R. Cracking Bitcoin wallets: I want what you have in the wallets, *Future Generation Computer Systems* **2019**, *91*, 136–143.
[11] Babu, P.R.; Bhaskari, D.L.; Satyanarayana, C. A comprehensive analysis of spoofing, *International Journal of Advanced Computer Science and Applications* **2010**, *1*(6).
[12] Jahangeer, A.; Bazai, S.U.; Aslam, S.; Marjan, S.; Anas, M.; Hashemi, S.H. A review on the security of IoT networks: From network layer's perspective, *IEEE Access* **2023**, *11*, 71073–71087.
[13] Abdulganiyu, O.H.; Ait Tchakoucht, T.; Saheed, Y.K. A systematic literature review for network intrusion detection system (IDS), *International Journal of Information Security* **2023**, *22*(5), 1125–1162.
[14] Khraisat, A.; Gondal, I.; Vamplew, P.; Kamruzzaman, J. Survey of intrusion detection systems: techniques, datasets and challenges, *Cybersecurity* **2019**, *2*(1), 20.
[15] Rakine, I.; Oukaira, A.; El Guemmat, K.; Atouf, I.; Ouahabi, S.; Talea, M.; Bouragba, T. Comprehensive review of intrusion detection techniques: ML and DL in different networks, *IEEE Access* **2025**, *13*, 104345–104367.
[16] Palshikar, A. What distinguishes binary from multi-class intrusion detection systems: Observations from experiments, *International Journal of Information Management Data Insights* **2022**, *2*(2), 100125.
[17] Mallidi, S.K.R.; Ramisetty, R.R. Advancements in training and deployment strategies for AI-based intrusion detection systems in IoT: A systematic literature review, *Discover Internet of Things* **2025**, *5*(1), 8.
[18] Arshad, J.; Azad, M.A.; Abdeltaif, M.M.; Salah, K. An intrusion detection framework for energy constrained IoT devices, *Mechanical Systems and Signal Processing* **2020**, *136*, 106436.
[19] Gholami, A.; Kim, S.; Dong, Z.; Yao, Z.; Mahoney, M.W.; Keutzer, K. A survey of quantization methods for efficient neural network inference. In *Low-Power Computer Vision*; Chapman and Hall/CRC, 2022; pp 291–326.
[20] Misrak, S.F.; Melaku, H.M. Lightweight intrusion detection system for IoT with improved feature engineering and advanced dynamic quantization, *Discover Internet of Things* **2025**, *5*(1), 97.
[21] El Husseini, F.; Noura, H.; Salman, O.; Chehab, A. Advanced machine learning approaches for zero-day attack detection: A review. In *2024 8th Cyber Security in Networking Conference (CSNet)*; IEEE, 2024; pp 297–304.
[22] Sabeel, U.; Heydari, S.S.; Elgazzar, K.; El-Khatib, K. Building an intrusion detection system to detect atypical cyberattack flows, *IEEE Access* **2021**, *9*, 94352–94370.
[23] Sahin, O.; Uludag, S. Leveraging inter-arrival time for efficient threat filtering: A parsimonious approach, *Computers & Security* **2025**, *154*, 104471.
[24] Toupas, P.; Chamou, D.; Giannoutakis, K.M.; Drosou, A.; Tzovaras, D. An intrusion detection system for multi-class classification based on deep neural networks. In *2019 18th IEEE International Conference on Machine Learning and Applications (ICMLA)*; IEEE, 2019; pp 1253–1258.

[25] Heidari, A.; Jabraeil Jamali, M.A. Internet of Things intrusion detection systems: a comprehensive review and future directions, *Cluster Computing* **2023**, *26*(6), 3753–3780.
[26] Mondal, S.; Khatun, A.; Moinuddin; Hassan, K.L.; Shieh, C.S. Quantized autonomous edge intrusion detection system for adaptive feature aware internet of medical things networks, *Discover Computing* **2026**, *29*(1), 392.
[27] Privacy-preserving detection of Wi-Fi deauthentication attacks on robotic systems using temporal traffic features, *IEEE Publications* **2026**.
[28] A hybrid feature fusion approach for multiclass Wi-Fi intrusion detection using classical machine learning, *Springer Nature* **2026**.
[29] Intrusion detection algorithm of wireless network based on network traffic anomaly analysis, *ScienceDirect* **2025**.
[30] Edge-friendly IoT intrusion detection using efficient boosted convolutional trees. In *Proceedings of IEEE Conference*; IEEE, 2025.
[31] An adaptive intrusion detection framework based on statistical modeling and protocol awareness, *IEEE Publications* **2025**.
[32] An energy-efficient intrusion detection system (IDS) tailored for resource-constrained edge computing environments, *IEEE Publications* **2025**.
[33] Chehade, A.; Ragusa, E.; Gastaldo, P.; Zunino, R. Hardware-aware neural architecture search for encrypted traffic classification on resource-constrained devices, *IEEE Transactions on Network and Service Management* **2026**.
[34] Yu, W.; Chen, Z.; Wang, H.; Miao, Z.; Zhong, D. Industrial network intrusion detection in open-set scenarios, *International Journal of Information Security* **2025**, *24*(1), 39.